

ESCUELA COLOMBIANA DE INGENIERIA JULIO GARAVITO

PLANIFICACION DE LAS CUATRO VERSIONES

The DOPO Hardest Game

DOPO 2026-1

Integrantes:

Madero - Valderrama

2026

1. Vision General del Proyecto

The DOPO Hardest Game es un videojuego de tipo arcade desarrollado en Java con arquitectura de capas (Presentacion / Dominio / Persistencia). El jugador controla un cuadrado de colores que debe cruzar un escenario lleno de enemigos moviles, recolectar monedas y llegar a la zona final antes de que se agote el tiempo.

El proyecto se desarrolla en cuatro versiones incrementales siguiendo el ciclo DOPO, integrando progresivamente los conceptos de Programacion Orientada a Objetos vistos en clase.

2. Requisitos que NO se van a Considerar

3. Version 0 — Estructura

Campo	Detalle
Objetivo	Definir la arquitectura base del proyecto y los primeros entregables de documentacion antes de escribir codigo de juego.
Temas POO aplicados	Abstraccion y modelado (Cap. 1), Clases y objetos basicos (Cap. 2 y 3), introduccion a relaciones entre objetos.
Entregables	Capa de Presentacion: boceto de interfaz grafica. Capa de Dominio: diagrama de clases inicial.
Fecha limite	Sabado 25 de abril

3.1 Mini-ciclos de la Version 0

Mini-ciclo	Descripcion	Resultado esperado
MC 0.1	Identificar las clases del dominio: Player, Enemy, Coin, GameState, LevelConfig, SafeZone, BoardEntity.	Lista de clases con atributos y responsabilidades definidas.
MC 0.2	Dibujar el diagrama de clases con relaciones de asociacion, composicion y dependencia.	Diagrama de clases exportado desde Astah.
MC 0.3	Boceto de la interfaz grafica: pantalla de menu, pantalla de juego con HUD y pantalla de victoria/derrota.	Mockup dibujado a mano
MC 0.4	Configurar el proyecto Maven en Eclipse con estructura de paquetes: domain, presentation, persistence, exceptions.	Proyecto compilable con estructura de carpetas correcta.

4. Version 1 — Presentacion + Dominio

Campo	Detalle
Objetivo	Implementar el juego jugable en modo un solo jugador con nivel basico: cuadrado rojo, enemigos azules que rebotan, monedas amarillas y logica completa de victoria/derrota.
Entregables	Codigo fuente de ambas capas, diagrama de clases actualizado, diagramas de secuencia, pruebas JUnit, retrospectiva del ciclo.

4.1 Mini-ciclos de la Version 1

Mini-ciclo	Descripcion	Resultado esperado
MC 1.1 — Dominio basico	Implementar Player (abstracta), RedPlayer, BoardEntity (interfaz), Enemy (abstracta), BasicBlueEnemy con rebote, Coin (abstracta) y YellowCoin.	Clases de dominio compilables con herencia e interfaces correctas.
MC 1.2 — GameState y logica	Implementar GameState con maquina de estados (PLAYING, PAUSED, WON, LOST), deteccion de colisiones jugador-enemigo, recoleccion de monedas y condicion de victoria.	Logica de juego funcionando sin interfaz grafica; validable con tests JUnit.
MC 1.3 — LevelConfig y configuracion	Implementar LevelConfig, SafeZone y LevelLoader para parsear archivos .txt. Crear level1.txt con 5 enemigos y 12 monedas.	Nivel cargable desde archivo con todas las entidades correctas.
MC 1.4 — Capa de presentacion	Implementar MainWindow, GamePanel GameLoop y MenuScreen con seleccion de modo y color de borde.	Juego visible y jugable con teclado WASD.

MC 1.5 — Excepciones y HUD	Implementar GameException con log automatico a game_errors.log. Agregar HUD con tiempo, muertes y monedas. Manejar errores de carga de nivel.	Errores registrados en log; HUD visible en pantalla.
MC 1.6 — Pruebas JUnit	Escribir pruebas unitarias para: tipos de jugador, colisiones, recoleccion de monedas, estados del juego, SafeZone y PlayerFactory.	pruebas con cobertura de los casos criticos del dominio.
MC 1.7 — Diagramas y retrospectiva	Exportar diagrama de clases desde Astah. Crear diagramas de secuencia	Documentacion completa del ciclo entregada.

5. Version 2 — Persistencia + Funcionalidades Avanzadas

Campo	Detalle
Objetivo	Agregar persistencia (guardar y abrir partidas), modo Player vs Player, multiples configuraciones de nivel y nuevos tipos de jugadores y enemigos.
Entregables	Codigo actualizado, planificacion de las cuatro versiones (este documento), objetivo de cada ciclo con mini-ciclos y requisitos descartados.

5.1 Mini-ciclos de la Version 2

Mini-ciclo	Descripcion	Resultado esperado
MC 2.1 — Nuevos tipos de jugador	Implementar BluePlayer (mas rapido y grande) y GreenPlayer (absorbe primer golpe, escudo). Actualizar PlayerFactory y MenuScreen.	Tres tipos de jugador seleccionables con comportamiento diferenciado.
MC 2.2 — Nuevos enemigos y elementos	Agregar FastBlueEnemy (doble velocidad), PatrolBlueEnemy (waypoints), Bomb (destruye al contacto) y LifeSource (vida extra). Actualizar LevelLoader.	Elementos especiales funcionando en nivel cargado desde archivo.
MC 2.3 — Multiples configuraciones	Crear level2.txt (laberinto con patrulleros y zona intermedia) y level3.txt (bombas y fuentes de vida). Agregar SkinCoin.	Tres niveles distintos seleccionables desde el menu.
MC 2.4 — Modo Player vs Player	Implementar GameMode.PVP: segundo jugador con teclas de flecha, colision P1 vs P2, condicion de victoria independiente por jugador.	Dos jugadores pueden competir en el mismo nivel.

MC 2.5 — Persistencia (guardar/abrir)	Implementar GameSaveManager: serializar GameState a archivo .properties y restaurarlo. Agregar opcion de carga en MenuScreen y guardado desde menu de ventana.	Partida guardada y restaurada correctamente con estado completo.
MC 2.6 — Integracion y pruebas	Actualizar GameTest con pruebas de los nuevos tipos. Verificar que todas las versiones anteriores siguen funcionando. Actualizar diagrama de clases.	Suite de pruebas extendida; juego estable con todas las funcionalidades.

6. Version 3 — Pulido y Entrega Final

Campo	Detalle
Objetivo	Corregir defectos encontrados en las versiones anteriores, mejorar la experiencia de juego, completar la documentacion tecnica y preparar la entrega final del proyecto.
Entregables	Version final del codigo, documentacion completa (diagramas actualizados, retrospectivas de todos los ciclos), presentacion del proyecto.

6.1 Mini-ciclos de la Version 3

Mini-ciclo	Descripcion	Resultado esperado
MC 3.1 — Correccion de defectos	Revisar y corregir bugs reportados en Version 1 y 2: colisiones imprecisas, comportamiento de SkinCoin al morir, visibilidad de P2 en modo single.	Juego sin defectos conocidos en todas las modalidades.
MC 3.2 — Mejoras de UI/UX	Refinar el HUD (indicador de escudo para GreenPlayer, icono de LifeSource), mejorar pantalla de victoria con estadisticas de la partida.	Interfaz mas informativa y pulida.
MC 3.3 — Documentacion final	Actualizar diagrama de clases, completar diagramas de secuencia faltantes, escribir retrospectivas de Version 2 y 3.	Documentacion tecnica completa y consistente con el codigo.
MC 3.4 — Preparacion de entrega	Limpiar el proyecto (remover codigo muerto, mejorar javadoc), generar JAR ejecutable, verificar que el proyecto corre desde cero en maquina limpia.	Entregable final listo para evaluacion.

